

Bit-Parallel Algorithm for the Constrained Longest Common Subsequence Problem

Sebastian Deorowicz*

Institute of Informatics

Silesian University of Technology

Akademicka 16, 44-100 Gliwice, Poland

sebastian.deorowicz@polsl.pl

Abstract. The problem of finding a constrained longest common subsequence (CLCS) for the sequences A and B with respect to the sequence P was introduced recently. Its goal is to find a longest subsequence C of A and B such that P is a subsequence of C . Most of the algorithms solving the CLCS problem are based on dynamic programming. Bit-parallelism is a technique of using single bits in a machine word for concurrent computation. We propose the first bit-parallel algorithm computing a CLCS and/or its length which outperforms the other known algorithms in terms of speed.

Keywords: constrained longest common subsequence, longest common subsequence, dynamic programming, bit-parallel algorithm

1. Introduction

In many fields there is a need to measure the similarity of sequences. Some of the most popular similarity measures are:

- edit distance, called sometimes Levenshtein distance—the lowest possible number of basic edit operations like insertion, deletion, substitution needed to transform one sequence to the other [20, 14],

*Address for correspondence: Silesian University of Technology, Akademicka 16, 44-100 Gliwice, Poland

- the length of a longest common subsequence of the sequences (a subsequence is obtained from a sequence by removal of zero or more symbols)—it is a dual of the so-called indel distance¹,
- sequence alignment—the sequences are aligned by insertion of gaps; the score of an alignment is calculated as a sum of scores of gaps and scores of aligned symbols of both sequences; the goal is to find an alignment of the highest score; the measure is especially popular in bioinformatics [14].

In this paper, we focus our attention on the length of the longest common subsequence (LCS) measure. It was noticed, that a direct application of the LCS measure to the protein comparison can lead to biologically useless results, since it treats the sequences as strings of letters and does not allow to use any prior biological knowledge of what is compared and what is known of the sequences. Therefore, a generalization of the LCS measure, the length of a *constrained longest common subsequence* (CLCS), was introduced by Tang *et al.* [27] and Tsai [28]. The authors postulate to use a third, so-called constrained, sequence and restrict to only such common subsequences that contain the third sequence as a subsequence. This allows to restrict the search to the subsequences of some predefined properties.

The LCS-computing problem is considered for about 30 years and there are many techniques to solve it. The, much younger, CLCS-computing problem looks very similar to the LCS, so it is natural to expect that the known LCS-computing algorithms can be modified to compute a CLCS. This appeared, however, harder than expected, so today there are only a few significantly different approaches to the CLCS problem [3, 6, 10, 18, 24, 25, 30].

One of the most powerful approaches for construction of fast algorithms in text processing field is bit-parallelism. It was invented by Dömölki [12] and reinvented by Baeza-Yates and Gonnet [4] (this work is much more known). The main idea is based on two simple observations. Firstly, the results of some computations can be stored in one bit only, e.g., in the pattern matching the result of the comparison of letters can be only “equal” or “unequal”. Secondly, the computations can be made simultaneously on many bits. The potential speedup is dependent on the machine word size, and since today 32- or 64-bit processors dominate, the gain can be large. There are, however, many drawbacks of this concept, e.g., *i*) one bit is an extremely small space and is sufficient for only some types of computations, *ii*) only some operations can be done simultaneously on many bits. From the above, it is not easy to provide the bit-parallel (BP) versions of the classical methods. As a “rule of thumb” the best candidates for bit-parallelization are the simplest algorithms in the field, e.g., the BP pattern matching algorithm is a version of the “naive” pattern matching algorithm [14].

There exist three bit-parallel algorithms for the LCS problem [1, 9, 17]. In most cases, the one by Hyrrö [17] is the fastest LCS-computing algorithm, but it (and also the two other) cannot be easily generalized for the CLCS problem. In this paper, we propose the application of the bit-parallel technique to the CLCS problem. The experiments show that in a typical case, the proposed method is a few times faster than the existing methods.

The paper is organized as follows. Section 2 contains the definitions. In Section 3, two important techniques for the LCS problem are presented. Especially, the BP method is presented in detail, since the knowledge of it is necessary to understand the method we propose. Section 4 contains a brief overview of the existing CLCS-computing algorithms. In Section 5, a BP algorithm computing the CLCS length is introduced. Then, we show how to extend the algorithm to obtain also a CLCS not only its length. Section 6 discusses the possible applications of the proposed algorithm in the multiple CLCS problem.

¹edit distance, in which only insertions and deletions are allowed

The following section contains the results of experimental comparison of existing algorithms. The last section concludes the paper.

2. Definitions

Let us have the sequences: $A = a_1a_2 \dots a_n$, $B = b_1b_2 \dots b_m$, and $P = p_1p_2 \dots p_r$, where $r \leq \min\{m, n\}$. Each of them is composed of symbols over an alphabet Σ of size σ . The *length (size)* of the sequence is the number of elements it contains. A sequence X' is a subsequence of X if it can be obtained from X by removing zero or more symbols. The LCS problem for sequences A and B is to find a longest possible sequence C being a subsequence of both A and B . The CLCS problem, being a generalization of the LCS problem for A, B, P , is to find a longest possible sequence C being a subsequence of A and B containing P as a subsequence.

The problem is symmetric, so it can be assumed without loss of generality that $m \leq n$. The pair (i, j) is called a *match* iff $a_i = b_j$. The triple (i, j, k) is called a *strong match* iff $a_i = b_j = p_k$. For simplicity, the notation X_s for $x_1x_2 \dots x_s$ is used. The rank of the triple (i, j, k) is the length of a CLCS for A_i, B_j, P_k .

The machine word size is denoted by w . The bit-parallel methods work on unsigned integers of size w and use bitwise operators: $\&$ (AND), $|$ (OR), $\wedge$ (exclusive OR), $\sim$ (negation of each bit). Also bit shifts are used: $\ll$ (shift to the left), $\gg$ (shift to the right). When the bits are shifted some of them are removed from the word and the empty positions are filled with 0's.

3. LCS Algorithms

3.1. Classical Dynamic Programming

In order to present the method proposed in this paper, it is helpful to describe the bit-parallel algorithm for a simpler, longest common subsequence, problem. A classical method to compute an LCS is to fill the two dimensional matrix according to the recurrence:

$$M(i, j) = \begin{cases} 0 & \text{for } i = 0 \text{ or } j = 0, \\ \max\{M(i-1, j), M(i, j-1)\} & \text{for } 0 < i \leq n, 0 < j \leq m, a_i \neq b_j, \\ M(i-1, j-1) + 1 & \text{for } 0 < i \leq n, 0 < j \leq m, a_i = b_j. \end{cases} \quad (1)$$

The value of $M(i, j)$ is the length of an LCS of sequences X_i and Y_j . Therefore, $M(n, m)$ stores the LCS length for the whole sequences (Figure 1). To obtain an LCS a traceback from the $M(n, m)$ to $M(1, 1)$ cell according to the cells used to compute the actual cell is performed. The symbols denoting the cells for which we have matches for the result read from the end (see Fig. 1 for example). This algorithm is of $O(mn)$ time complexity, which was proved to be the lower bound in the comparison model [31].

There are a few algorithms which are faster in practice, unfortunately, in the worst case they are still $\Omega(mn)$ -time. In the RAM computation model, there are faster methods even in the asymptotic sense. The examples are the algorithms by Masek–Paterson [23] and Crochemore *et al.* [8], both needing $O(mn/\log n)$ time. They are, however, rather impractical due to a large constant hidden in the big- O notation. The best known lower bound of time complexity in this model is $\Omega(n \log n)$ [15].

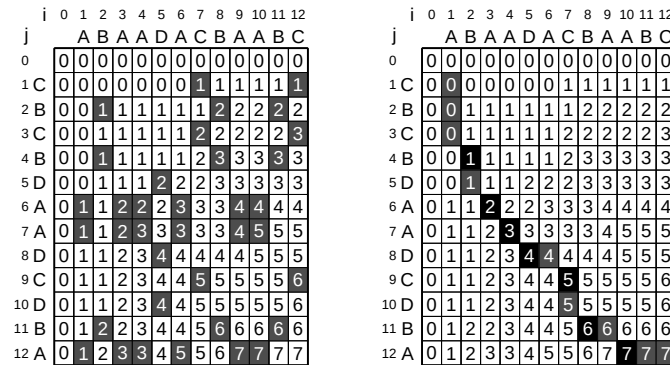


Figure 1. An example of the classical dynamic programming algorithm computing the LCS for the sequences: ABAADACBAABC, CBCBDAADCDBA. The marked cells denote matches. Left: the DP matrix with marked matches. Right: the traceback; the black cells represents the matches taken to the result: BAADCBA.

One of two most practical algorithm categories for the LCS problem are these based on the Hunt–Szymanski scheme [16], which restricts the computations to matches only. Since the number of matches is usually much less than mn , this is a significantly faster method than processing all the matrix cells. A good introduction to the LCS computing algorithms can be found in [14, 2]. In the experimental survey by Bergroth *et al.* [5], the method by Kuo and Cross [19] based on the Hunt–Szymanski scheme proved the fastest algorithm.

The second category of very fast algorithms for the LCS problem are bit-parallel methods at which we pay a bit more attention in the next section.

3.2. Bit-Parallel Technique for the LCS Problem

While looking at the dynamic programming (DP) matrix (Figure 1) one can notice a nice (from the bit-parallelization point of view) property: neighbor cells differ by at most one. This observation lays at the ground of the LCS-computing BP algorithms, since instead of storing the cell value, only the information (in the form of 1 bit) whether the cell is greater than one, say the upper one, of its neighbors can be stored. Moreover, the bits of successive rows can be computed simultaneously. The three bit-parallel algorithms for the LCS problem were presented by Allison and Dix [1], Crochemore *et al.* [9], and Hyvärö [17]. They are similar to each other (the discussion of the differences can be found in [17]), so we describe here only the most recent (Figure 2), which is typically the fastest in practice.

Figure 3 presents the computation of the LCS length for the same sequences as in the previous example. The DP-matrix given at the rightmost part of the figure is only for the illustration and is not maintained by the algorithm. The algorithm works on bit vectors (for simplicity we assume here that a vector fits a single machine word, but it is easy to modify the algorithm to work on arrays of machine words). The frame MASKS contains σ vectors (one for each alphabet symbol) computed in a preprocessing stage (lines 01–02 of the pseudocode). The j th bit of the vector related to the $c \in \Sigma$ symbol is set to 1 iff $b_j = c$. The working vector of the algorithm is V . It reflects the changes of values in a single column of the dynamic programming (DP) matrix, i.e., the j th bit of V is 0 iff $M(i, j) - M(i, j - 1) = 1$ for the actual, i th, column. Equivalently, after processing the i th column the bit index j of sth 0 in V is

BitParLCS()

```

{Preprocessing}
01 for  $c \in \Sigma$  do  $Y_c \leftarrow 0^m$ 
02 for  $j \leftarrow 1$  to  $m$  do  $Y_{b_j} \leftarrow Y_{b_j} | 0^{m-j}10^j$ 
{Computing LCS}
03  $V \leftarrow 1^m$ 
04 for  $i \leftarrow 1$  to  $n$  do
05    $U \leftarrow V \& Y_{a_i}$ 
06    $V \leftarrow (V + U) | (V - U)$ 
{Calculating number of 0's in  $V$ }
07  $\ell \leftarrow 0$ ;  $V \leftarrow \sim V$ 
08 while  $V \neq 0^m$  do
09    $\ell \leftarrow \ell + 1$ ;  $V \leftarrow V \& (V - 1)$ 
10 return  $\ell$ 

```

Figure 2. Bit-parallel algorithm computing the LCS length by Hyrrö [17]

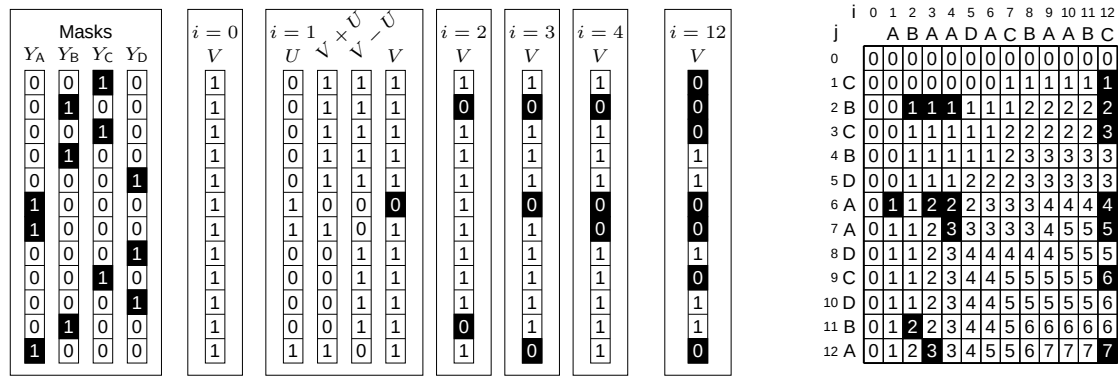


Figure 3. An example of the bit-parallel algorithm by Hyrrö for the LCS problem.

the smallest possible index of row, such that (i', j) is a match of rank s and $i' \leq i$. The successive frames ($i = 0, i = 1, \dots, i = 4, i = 12$) show the contents of V after processing the i th column. For $i = 1$ more details are presented reflecting the computations made in lines 05–06 of the pseudocode. The 0's in the vectors V and the cells of the columns of M at which the value increases are marked to show the relation between the columns of M and the vector V . In lines 07–09, the number of 0's in the final V vector is computed, since this is the LCS length.

The main loop of the algorithm is executed n times and inside the loop there are only some bit operations. If $m > w$, the bit vectors must be emulated as an $\lceil m/w \rceil$ -element array of machine words. The cost of bit computations on a vector is proportional to the array size, so the total time complexity is $O(n \lceil m/w \rceil)$.

To obtain an LCS of length ℓ a clever backtracking on bits is necessary. The main idea is to store bit-vector V for each processed column ($V(i)$ denotes the content of V after processing i th column) and treat the bit values as “signposts”. If j th bit of $V(i)$ is 1 then $M(i, j) = M(i, j - 1)$, so we can move from cell (i, j) to $(i, j - 1)$ since the ranks of these cells are the same. Otherwise, we move to

BitParLCS-Result()

```

{ $\ell$  is the result given by BitParLCS()}
01   $\ell^* \leftarrow \ell$ ;   $i \leftarrow n$ ;   $j \leftarrow m$ 
02  while  $\ell^* > 0$  do
03      if  $a_i = b_j$  then
04           $s_{\ell^*} \leftarrow a_i$ ;   $\ell^* \leftarrow \ell^* - 1$ 
05      else
06          if  $j$ th bit of  $V(i)$  is 1 then
07               $j \leftarrow j - 1$ 
08          else
09               $i \leftarrow i - 1$ 
10  return  $s_1 s_2 \dots s_{\ell}$ 

```

Figure 4. Bit-parallel algorithm recovering an LCS from the matrix produced by BitParLCS() algorithm

	i	0	1	2	3	4	5	6	7	8	9	10	11	12
j		A	B	A	A	D	A	C	B	A	A	B	C	
0		0	0	0	0	0	0	0	0	0	0	0	0	0
1 C		0	0	0	0	0	0	1	1	1	1	1	1	1
2 B		0	0	1	1	1	1	1	2	2	2	2	2	2
3 C		0	0	1	1	1	1	1	2	2	2	2	2	3
4 B		0	0	1	1	1	1	1	2	3	3	3	3	3
5 D		0	0	1	1	1	2	2	2	3	3	3	3	3
6 A		0	1	1	2	2	2	3	3	3	4	4	4	4
7 A		0	1	1	2	3	3	3	3	3	4	5	5	5
8 D		0	1	1	2	3	4	4	4	4	4	5	5	5
9 C		0	1	1	2	3	4	4	5	5	5	5	6	6
10 D		0	1	1	2	3	4	4	5	5	5	5	6	6
11 B		0	1	2	2	3	4	4	5	6	6	6	6	6
12 A		0	1	2	3	3	4	5	5	6	7	7	7	7

	i	0	1	2	3	4	5	6	7	8	9	10	11	12
j		A	B	A	A	D	A	C	B	A	A	B	C	
0														
1 C		1	1	1	1	1	1	0	0	0	0	0	0	0
2 B		1	1	0	0	0	0	0	1	0	0	0	0	0
3 C		1	1	1	1	1	1	0	1	1	1	1	0	0
4 B		1	1	1	1	1	1	1	0	0	0	0	1	0
5 D		1	1	1	1	0	0	1	1	1	1	1	1	0
6 A		1	0	1	0	0	1	0	0	1	0	0	0	0
7 A		1	1	1	1	0	0	1	1	1	1	0	0	0
8 D		1	1	1	1	1	0	0	0	0	1	1	1	1
9 C		1	1	1	1	1	1	0	0	0	1	1	0	0
10 D		1	1	1	1	1	1	1	1	1	1	1	1	1
11 B		1	1	0	1	1	1	1	1	1	0	0	0	1
12 A		1	1	1	0	1	1	0	1	1	0	0	0	0

Figure 5. An example of the bit-parallel algorithm by Hyrrö for the LCS problem. The output sequence is BAADCBA. Left: DP matrix just for comparison of what is going on. Right: backtracking on bit matrix. Note that for simplicity of presentation the bits in Hyrrö's algorithm are numbered from 1

cell $(i - 1, j)$, since it has a not-less value of the left and upper neighbor of (i, j) . Of course, if (i, j) is a match we move diagonally to $(i - 1, j - 1)$ and output the current symbol. The pseudocode of this procedure is given in Figure 4 and an illustrating example in Figure 5. The worst-case time complexity of this procedure is $O(n + m) = O(n)$ since in each loop iteration we decrease at least one index.

4. Existing Algorithms for the CLCS Problem

The existing CLCS computing algorithms are enumerated in Table 1. The first proposed method by Tsai [28] was completely impractical due to its large time and space complexities. Almost immediately a number of much faster algorithms were proposed. The algorithms by Peng [24], Chin *et al.* [6], Peng and Ting [25] compute the three dimensional dynamic programming matrix and the time complexity of them is a product of the sequence lengths. The algorithms by Arslan-Eğecioğlu [3] and Wang [30] restrict the computations to only that part of the matrix that may influence on the final result, but, in the worst case, they also require $O(rnm)$ time.

Authors	Year	Time	Space
Tsai [28]	2003	$O(rn^2m^2)$	$O(n^2m^2)$
Peng [24]	2003	$O(rnm)$	$O(rm)$
Chin <i>et al.</i> [6]	2004	$O(rnm)$	$O(rm)$
Peng, Ting [25]	2004	$O(rnm)$	$O(rn)$
Arslan, Egecioğlu [3]	2005	$O(rnm)$	$O(nm)$
Wang [30]	2006	$O(rnm)$	$O(rm)$
Deorowicz [10]	2007	$O(r(m\ell + d) + n)$	$O(\max(n, d))$
Illiopulos, Rahman [18]	2008	$O(rd \log \log n + n)$	$O(\max(n, d))$
Deorowicz, Obstój [11]	2010	$O(r(m\ell + d) + n)$	$O(\max(n, d))$

Table 1. Time and space complexities of the CLCS length computing algorithms (d means the total number of matches between A and B , and ℓ means the length of an LCS for A and B)

The algorithm by Deorowicz [10] is based on the Hunt–Szymanski (HS) idea [16] for the LCS problem. The implementation of this approach is not direct due to the CLCS properties, so the time complexities are not so good as one could expect knowing the HS algorithm for the LCS problem. In the worst case, the time complexity of the algorithm is also of $O(rnm)$. Nevertheless, the complexity expressed in terms of the output size and some properties of the sequences, $O(r(m\ell + d) + n)$, seems to be promising, since $\ell \leq n$ and $d \leq nm$, where ℓ is the length of an LCS for A and B and d is the total number of matches between A and B . For typical cases it makes significantly smaller number of computations than the DP-based algorithms [11]. The algorithm by Illiopulos and Rahman [18] in the worst case is worse than $O(rnm)$, but for typical cases is better. Unfortunately, it employs sophisticated data structures and tends to be slow in practice.

A more detailed overview of the CLCS algorithms together with the experimental comparison of their practical efficiency can be found in [11].

5. Proposed Algorithm

5.1. Preliminaries

There are important differences between the CLCS and LCS problems, which make the application of the bit-parallel technique hard and in consequence no such algorithm for the CLCS problem has been presented so far. Similarly as for the LCS problem, the point of departure for bit-parallelization will be the conceptually simplest algorithm, i.e., the algorithm by Chin *et al.* [6]. It computes the three dimensional array M using the recurrence:

$$M(i, j, k) = \begin{cases} M(i-1, j-1, k-1) + 1 & \text{if } i, j, k > 0 \text{ and } a_i = b_j = p_k, \\ M(i-1, j-1, k) + 1 & \text{if } i, j > 0, a_i = b_j \text{ and } (k = 0 \text{ or } a_i \neq p_k), \\ \max\{M(i-1, j, k), M(i, j-1, k)\} & \text{if } i, j > 0 \text{ and } a_i \neq b_j. \end{cases} \quad (2)$$

k=0														
j	i	0	1	2	3	4	5	6	7	8	9	10	11	12
		A	B	A	A	D	A	C	B	A	A	B	C	
0		0	0	0	0	0	0	0	0	0	0	0	0	0
1 C		0	0	0	0	0	0	1	1	1	1	1	1	1
2 B		0	0	1	1	1	1	1	2	2	2	2	2	2
3 C		0	0	1	1	1	1	2	2	2	2	2	2	3
4 B		0	0	1	1	1	1	2	3	3	3	3	3	3
5 D		0	0	1	1	1	2	2	2	3	3	3	3	3
6 A		0	1	1	2	2	2	3	3	3	4	4	4	4
7 A		0	1	1	2	3	3	3	3	3	4	5	5	5
8 D		0	1	1	2	3	4	4	4	4	5	5	5	5
9 C		0	1	1	2	3	4	4	5	5	5	5	5	6
10 D		0	1	1	2	3	4	4	5	5	5	5	5	6
11 B		0	1	2	2	3	4	4	5	6	6	6	6	6
12 A		0	1	2	3	3	4	5	5	6	7	7	7	7

k=1														
j	i	0	1	2	3	4	5	6	7	8	9	10	11	12
		A	B	A	A	D	A	C	B	A	A	B	C	
0		-	-	-	-	-	-	-	-	-	-	-	-	-
1 C		-	-	-	-	-	-	1	1	1	1	1	1	1
2 B		-	-	-	-	-	-	1	2	2	2	2	2	2
3 C		-	-	-	-	-	-	2	2	2	2	2	3	3
4 B		-	-	-	-	-	-	2	3	3	3	3	3	3
5 D		-	-	-	-	-	-	2	3	3	3	3	3	3
6 A		-	-	-	-	-	-	2	3	4	4	4	4	4
7 A		-	-	-	-	-	-	2	3	4	5	5	5	5
8 D		-	-	-	-	-	-	2	3	4	5	5	5	5
9 C		-	-	-	-	-	-	5	5	5	5	5	6	6
10 D		-	-	-	-	-	-	5	5	5	5	5	6	6
11 B		-	-	-	-	-	-	5	6	6	6	6	6	6
12 A		-	-	-	-	-	-	5	6	7	7	7	7	7

k=2														
j	i	0	1	2	3	4	5	6	7	8	9	10	11	12
		A	B	A	A	D	A	C	B	A	A	B	C	
0		-	-	-	-	-	-	-	-	-	-	-	-	-
1 C		-	-	-	-	-	-	-	-	-	-	-	-	-
2 B		-	-	-	-	-	-	-	2	2	2	2	2	2
3 C		-	-	-	-	-	-	-	2	2	2	2	3	3
4 B		-	-	-	-	-	-	-	3	3	3	3	3	3
5 D		-	-	-	-	-	-	-	3	3	3	3	3	3
6 A		-	-	-	-	-	-	-	3	4	4	4	4	4
7 A		-	-	-	-	-	-	-	3	4	5	5	5	5
8 D		-	-	-	-	-	-	-	3	4	5	5	5	5
9 C		-	-	-	-	-	-	-	3	4	5	5	6	6
10 D		-	-	-	-	-	-	-	3	4	5	5	6	6
11 B		-	-	-	-	-	-	-	6	6	6	6	6	6
12 A		-	-	-	-	-	-	-	6	7	7	7	7	7

k=3														
j	i	0	1	2	3	4	5	6	7	8	9	10	11	12
		A	B	A	A	D	A	C	B	A	A	B	C	
0		-	-	-	-	-	-	-	-	-	-	-	-	-
1 C		-	-	-	-	-	-	-	-	-	-	-	-	-
2 B		-	-	-	-	-	-	-	-	-	-	-	-	-
3 C		-	-	-	-	-	-	-	-	-	-	-	-	-
4 B		-	-	-	-	-	-	-	-	-	-	-	-	-
5 D		-	-	-	-	-	-	-	-	-	-	-	-	-
6 A		-	-	-	-	-	-	-	-	-	-	-	-	-
7 A		-	-	-	-	-	-	-	-	-	-	-	-	-
8 D		-	-	-	-	-	-	-	-	-	-	-	-	-
9 C		-	-	-	-	-	-	-	-	-	-	-	-	-
10 D		-	-	-	-	-	-	-	-	-	-	-	-	-
11 B		-	-	-	-	-	-	-	-	-	-	-	-	-
12 A		-	-	-	-	-	-	-	-	-	-	-	-	-

Figure 6. An example of the CLCS computing algorithm by Chin *et al.* [6]. The sequences are: $A = \text{ABAADACBAABC}$, $B = \text{CBCBDAADCDDBA}$, $P = \text{CBB}$. Black cells denote strong matches

The boundary conditions are:

$$\begin{aligned}
 M(i, 0, k) &= 0, & \text{for } 0 \leq i \leq n, k = 0 \\
 M(0, j, k) &= 0, & \text{for } 0 \leq j \leq m, k = 0 \\
 M(i, 0, k) &= -\infty, & \text{for } 0 \leq i \leq n, 1 \leq k \leq r, \\
 M(0, j, k) &= -\infty, & \text{for } 0 \leq j \leq m, 1 \leq k \leq r.
 \end{aligned}$$

An example of the algorithm in work is given in Figure 6. Similarly to the LCS problem, $M(i, j, k)$ stores the length of a CLCS for sequences A_i , B_j , P_k .

The **two most important difficulties** of bit-parallelization of this algorithm are:

1. the differences between neighbor cells of M can be larger than 1, so, in general, require more than one bit to be stored,
2. there are $r + 1$ levels and the levels are not independent, since a value for a strong match is computed according to the value of a cell at the previous level.

These problems are caused by appearance of strong matches. If (i, j, k) is a strong match, $M(i, j, k)$ is one more than the length of a CLCS for A_{i-1} , B_{j-1} when P_{k-1} is constrained and $M(i, j-1, k)$ is the length of a CLCS for A_i , B_{j-1} when a longer sequence, P_k , is constrained. Therefore, the difference between $M(i, j, k)$ and $M(i, j-1, k)$ can be large. The constrained sequence influences significantly the CLCS length, which can be seen, e.g., in Figure 6 when the values of (i, j, k) for fixed i, j and various k are compared.

If (i, j, k) is not a strong match, then $M(i, j, k) - M(i, j-1, k) \in \{0, 1\}$ if $b_j \neq p_k$ and $M(i, j, k) - M(i-1, j, k) \in \{0, 1\}$ if $a_i \neq p_k$. This is due to the fact that for all the mentioned cells, the constrained sequence is the same and truncating one main sequence (A or B) by only one symbol cannot make the length of a CLCS for them shorter by more than 1.

The main concept of the algorithm introduced in this paper is to compute explicitly only the cells that can differ by more than 1 from their upper neighbor and compute the rest using a fast BP technique. The cells that are explicitly computed belong to the rows containing strong matches.

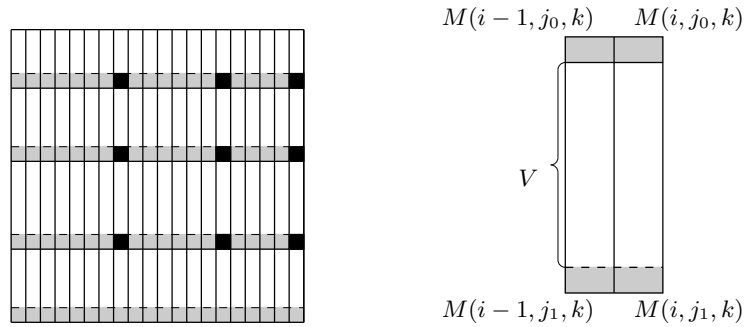


Figure 7. An idea of the strip-based bit-parallel algorithm for the CLCS problem. The grayed rows contain strong matches (black boxes) and the values for these rows are computed explicitly. The vertical strips ending at grayed rows are computed using BP technique.

5.2. Bit-Parallel Strip Algorithm

Our algorithm computes the CLCS length in a similar way like the BP algorithms compute the LCS length (Section 3.2). We maintain matrix M , in which values for some cells are explicitly stored. It will be shown later how this 3-D matrix can be implemented with **two FIFO queues**.

Matrix M is processed level-wise. Let us consider a single level for $k > 0$. The algorithm splits the nonnegative part of two dimensional matrix into, **so-called, strips**. A strip is a rectangular area of a level of matrix M of **width 1 column and variable height**. It is defined by a 4-tuple $\langle i, j_0, j_1, k \rangle$. The vertical placement of strips is determined by strong match positions. Each strip ends at the row denoted by p_k symbol (i.e., for such j_1 that $b_{j_1} = p_k$) and starts just after previous such a row of index j_0 . For completeness, some strips end at the bottom boundary of the matrix even if $b_m \neq p_k$. The cells of the uppermost nonnegative row are outside the strips. The only position within the strip where a strong match can appear is its end, so it is possible to maintain bit vector $V_{j_0}(i)$ representing changes of values from range $M(i, j_0, k), \dots, M(i, j_1 - 1, k)$. The 0s in $V_{j_0}(i)$ represent (similarly to the LCS-computing BP algorithm) the smallest possible row indexes of matches of successive ranks in the processed part of matrix M . It is easy to find the part of matrix M , which contains $-\infty$ values [11], so for each level the procedure FindStartingPoint (Figure 8) is used to exclude that part of the matrix from computations.

The computation of each level is performed strip-by-strip in a column-wise and within a column in a row-wise order. An essential part of the algorithm is the computation of a single strip when the next to the left and upper strips are already known. For each strip $\langle i, j_0, j_1, k \rangle$ the following values are updated:

- $V_{j_0}(i)$ —bit vector representing at successive bits the information whether $M(i, j, k) = M(i, j - 1, k)$ (bit value 1) or $M(i, j, k) \neq M(i, j - 1, k)$ (bit value 0) for $j_0 < j < j_1$,
- $C_{j_0}(i)$ —number of 0s in vector $V_{j_0}(i)$,
- $M(i, j_1, k)$ —the CLCS length for sequences A_i , B_{j_1} , and P_k .

Lemma 5.1. The algorithm presented in Figure 9 properly computes $V_{j_0}(i)$, $C_{j_0}(i)$, $M(i, j_1, k)$, and the values $M(i, j, k)$ for all $j_0 < j \leq j_1$ such that $(i + 1, j + 1, k + 1)$ is a strong match if the following

FindStartingPoint(k)

```

01   $i^* \leftarrow 1; \quad k^* \leftarrow 1$ 
02  while  $i^* \leq n$  and  $k^* \leq k$  do
03      if  $a_{i^*} = p_{k^*}$  then
04           $k^* \leftarrow k^* + 1$ 
05       $i^* \leftarrow i^* + 1$ 
06  if  $k^* \leq k$  then
07      return  $n + 1, m + 1$ 
08   $j^* \leftarrow 1; \quad k^* \leftarrow 1$ 
09  while  $j^* \leq m$  and  $k^* \leq k$  do
10      if  $b_{j^*} = p_{k^*}$  then
11           $k^* \leftarrow k^* + 1$ 
12       $j^* \leftarrow j^* + 1$ 
13  if  $k^* \leq k$  then
14      return  $n + 1, m + 1$ 
15  return  $i^* - 1, j^* - 1$ 

```

Figure 8. Finding starting point for the level k

values are known: $M(i - 1, j_0, k)$, $M(i, j_0, k)$, $V_{j_0}(i - 1)$, $C_{j_0}(i - 1)$, and $M(i - 1, j_1 - 1, k - 1)$ if (i, j_1, k) is a strong match.

Proof:

The value $M(i - 1, j_0, k)$ is the largest rank of matches (i', j', k) for $0 < i' \leq i - 1$ and $0 < j' \leq j_0$. Each s th 0, for all valid s , in vector $V_{j_0}(i - 1)$ at bit position j (counting from j_0) is the smallest row index of match of rank $M(i - 1, j_0, k) + s$ for matches (i', j', k) when $0 < i' \leq i - 1$ and $0 < j' \leq j_1$.

The j th bit of vector Y_{a_i, j_0} (part of Y_{a_i} starting at j_0 and ending at j_1 bits) is 1 iff $a_i = b_j$. The bit processing (lines 01–02) are the same as in the Hyyrö's [17] algorithm (Section 3.2). In line 03, the number of 0s in vector $V_{j_0}(i)$ is calculated (it can be equal to $C_{j_0}(i - 1)$ or larger by 1 if a new 0 in vector $V_{j_0}(i)$ is encountered). Therefore, after these lines, 0s in $V_{j_0}(i)$ represent the row indexes of matches of ranks starting from $M(i - 1, j_0, k)$. $M(i, j_0, k)$ (the largest rank of matches (i', j', k) for $0 < i' \leq i$ and $0 < j' \leq j_0$) can be, however, larger than $M(i - 1, j_0, k)$. E.g., if (i, j_0, k) is a strong match then $M(i, j_0, k) - M(i - 1, j_0, k)$ can be even larger than 1. Therefore, the $M(i, j_0, k) - M(i - 1, j_0, k)$ least significant 0s from $V_{j_0}(i)$ are removed to preserve that the matches of $V_{j_0}(i)$ reflect the ranks starting from $M(i, j_0, k)$ (line 04). Accordingly $C_{j_0}(i)$ is updated.

Then, the value $M(i, j_1, k)$ is computed. If (i, j_1, k) is not a strong match it is computed as a sum of $M(i, j_0, k)$ and the number of 0s in the actual vector V . Otherwise, it is calculated according to the value $M(i - 1, j - 1, k - 1)$ from the previous level (lines 05–08).

Finally, the values $M(i, j, k)$, $j_0 < j \leq j_1$, are calculated from $M(i, j_0, k)$ and the actual vector V if $(i + 1, j + 1, k + 1)$ are strong matches (these values of M are necessary for computing the next level) (lines 09–10). $\square$

The computation of the CLCS length is level-wise. Let us consider computation of a complete level for $k > 0$ using the procedure for strip computation. The case of $k = 0$ will be discussed later.

ComputeStrip(i, j_0, j_1, k)

```

01   $U \leftarrow V_{j_0}(i-1) \& Y_{a_i, j_0}$ 
02   $V_{j_0}(i) \leftarrow (V_{j_0}(i-1) + U) \mid (V_{j_0}(i-1) - U)$ 
03   $C_{j_0}(i) \leftarrow C_{j_0}(i-1) + \text{was\_new\_0\_produced\_in\_previous\_line}$ 
04  Remove  $M(i, j_0, k) - M(i-1, j_0, k)$  least significant 0s from  $V_{j_0}(i)$  and update  $C_{j_0}(i)$  accordingly
05  if  $(i, j_1, k)$  is a strong match then
06       $M(i, j_1, k) \leftarrow M(i-1, j_1-1, k-1) + 1$ 
07  else
08       $M(i, j_1, k) \leftarrow \max(M(i, j_0, k) + C_{j_0}(i), M(i-1, j_1, k))$ 
09  for all strong matches  $(i+1, j+1, k+1)$  where  $j \in [j_0, j_1]$  do
10      Compute  $M(i, j, k)$  on  $M(i, j_0, k)$  and  $V_{j_0}(i)$ 

```

Figure 9. Bit-parallel strip computing algorithm

Lemma 5.2. Assuming that for all strong matches (i, j, k) the values $M(i-1, j-1, k-1)$ are correctly computed, the algorithm computing a single level $k > 0$ of matrix M (Figure 10) correctly determines the values $M(i, j, k)$ for such triples that $(i+1, j+1, k+1)$ is a strong match.

Proof:

Let the loop invariant of the main loop of the algorithm (lines 18–20) be:

- $M(i, j, k)$ stores the largest rank of matches (i', j', k) such that $0 < i' \leq i$, $0 < j' \leq j$, when $b_j = p_k$ or $j = m$.

According to the recurrence by Chin *et al.* (Equation 2), all matrix cells $M(i', j', k)$ for $i' < i^*$ or $j' < j^*$ are non-positive. For every triple (i', j^*, k) , such that $i^* < i' \leq n$ holds:

- if the triple is a strong match, the value $M(i', j^*, k)$ is properly calculated on the previous level,
- otherwise the value $M(i', j^*, k)$ is calculated as $\max(M(i'-1, j^*, k), M(i', j^*-1, k)) = M(i'-1, j^*, k)$ exactly as in the recurrence by Chin.

Therefore, the values of M for the top boundary are properly calculated (lines 05–09).

The values for strong matches laying at the left boundary, i.e., (i^*, j, k) are calculated on the $(k-1)$ th level of M (lines 10–17). Similarly, as for the top boundary all the values $M(i^*, j', k)$ for non-matches are equal to $M(i^*, j_0, k)$ for the largest $j_0 < j'$ such that (i^*, j_0, k) is a strong match (actually they are not computed, but we know their values).

For each j_0 such that (i^*, j_0, k) is a strong match and the smallest $j_1 > j_0$ such that (i, j_1, k) is also a strong match or $j_1 = m$, the 4-tuple $\langle i, j_0, j_1, k \rangle$ defines a strip. Vectors $V_{j_0}(i^*)$ contain no 0s, since the values of M for the strip (except for j_1) are equal.

Before the main loop (lines 18–20) the loop invariant is true for $i = i^*$ and $j = j^*$ (the initial values of i and j). Assuming that $M(i-1, j_0, k)$, $M(i, j_0, k)$, $M(i-1, j_1, k)$, and $V_{j_0}(i-1)$ are correct, the algorithm ComputeStrip (Figure 9) correctly computes $M(i, j_1, k)$ and $V_{j_0}(i)$. It also computes all the cells $M(i, j, k)$ such that $(i+1, j+1, k+1)$ are strong matches. $\square$

```

ComputeLevel( $k$ )
01   $i^*, j^* \leftarrow \text{FindStartingPoint}(k)$ 
02  if  $i^* > n$  then
03    return false
04  Precompute bit masks for level  $k$ 
    {Computing top boundary}
05  for  $i \leftarrow i^*$  to  $n$  do
06    if  $a_i = p_k$  then
07       $M(i, j^*, k) \leftarrow M(i - 1, j^* - 1, k - 1) + 1$ 
08    else
09       $M(i, j^*, k) \leftarrow M(i - 1, j^*, k)$ 
    {Computing left boundary}
10   $y \leftarrow 0$ ;  $j_0 \leftarrow j^*$ ;  $\text{strips} \leftarrow \langle \rangle$ 
11  for  $j \leftarrow j^* + 1$  to  $m$  do
12     $y \leftarrow (y \ll 1) \mid 1$ 
13    if  $b_j = p_k$  or  $j = m$  then
14       $\text{strips} \leftarrow \text{strips} + \langle j_0, j \rangle$ 
15       $V_{j_0}(i^*) \leftarrow y$ ;  $C_{j_0}(i^*) \leftarrow 0$ 
16       $M(i^*, j_0, k) \leftarrow M(i^* - 1, j_0 - 1, k - 1) + 1$ 
17       $j_0 \leftarrow j$ ;  $y \leftarrow 0$ 
    {Main computing}
18  for  $i \leftarrow i^* + 1$  to  $n$  do
19    for all strips  $\langle j_0, j_1 \rangle$  in  $\text{strips}$  do
20      ComputeStrip( $i, j_0, j_1, k$ )
21  return true

```

Figure 10. Bit-parallel level computing algorithm

Computing the level $k = 0$ is a special case, since there is no strong matches at it. Therefore, in fact the BP LCS-computing of this level can be used. The only exception is that values $M(i, j, 0)$ need to be computed explicitly if $(i + 1, j + 1, 1)$ are strong matches. We use therefore ComputeStrip algorithm and define for completeness that no triple $(i, j, 0)$ is a strong match.

Lemma 5.3. The algorithm computing level $k = 0$ of matrix M (Figure 11) correctly computes the values of M for such triples that $(i + 1, j + 1, 1)$ is a strong match.

Proof:

The values of $M(i, 0, 0)$ for $0 \leq i \leq n$ and $M(0, j, 0)$ for $0 \leq j \leq m$ are set to 0 according to the recurrence by Chin (Equation 2). Since there are no strong matches for $k = 0$, the strips are defined by 4-tuples $\langle i, 0, m, 0 \rangle$. Vector $V_1(0)$ contains only 1s since all the values of M for the left boundary (the 0th column) are equal. By the same arguments as in the proof of Lemma 5.2 this lemma is true. $\square$

The last thing is to collect all parts together obtaining the algorithm ComputeCLCS presented in Figure 12.

Lemma 5.4. The bit-parallel CLCS-length-computing algorithm (Figure 12) is correct.

ComputeLevel0(k)

```

{Precomputing}
01  for  $i \leftarrow 0$  to  $n$  do
02       $M(i, 0, 0) \leftarrow 0$ 
03  for  $j \leftarrow 1$  to  $m$  do
04       $M(0, j, 0) \leftarrow 0$ 
05   $V_1(0) \leftarrow 1^m$ 
{Main computing}
06  for  $i \leftarrow 1$  to  $n$  do
07      ComputeStrip( $i, 1, m, 0$ )

```

Figure 11. Bit-parallel level-0 computing algorithm

ComputeCLCS()

```

01  ComputeLevel0()
02  for  $k \leftarrow 1$  to  $r$  do
03      if ComputeLevel( $k$ ) = false then
04          return 0
05  if  $b_m = p_r$  then
06      return  $M(n, m, r)$ 
07  else
08       $j' \leftarrow \max\{j : b_j = p_r \wedge 1 \leq j \leq m\}$ 
09      return  $M(n, j', r) + C_{j'+1}(n)$ 

```

Figure 12. Bit-parallel CLCS computing algorithm

Proof:

We follow by recurrence on k , the level number. The algorithm computing level 0 is correct. Assuming that the level $k - 1$ is correctly computed, the level k is also correct. Therefore, the cell $M(n, m, r)$ stores the length of a CLCS for A, B, P if $b_m = p_r$. If the last row of matrix M at level r does not contain strong matches, the result is computed as the sum of the largest value of M at level r and the number of 0s at the last strip, i.e., $\langle n, j', m, k \rangle$. If there is no subsequence satisfying the requirements, the algorithm returns 0. $\square$

An example of the algorithm in work is presented in Figure 13. The calculations illustrate the case of $k = 1$. The upper part of the figure shows the contents of matrix M . In fact, the only cells computed explicitly are these that are gray or black. Apart from level $k = 1$ also levels for $k = 0$ (strong matches are computed relating on values from this level) and $k = 2$ (some cells from level $k = 1$ must be explicitly computed for level $k = 2$) are shown. Nevertheless, these matrix levels are only for presentation, since the structures maintained by the algorithm are presented at the lower part of the figure.

Frame MASKS contains the bit vectors for strip rows. The processing of the level starts from the point $(7, 1)$. Frame $i = 7$ shows the computations made while processing the 7th column. This is the left boundary, so only values of strong matches $((7, 1, 1), (7, 3, 1), (7, 9, 1))$ are calculated and the vectors V are initialized. Moreover, $M(7, 10, 1)$ is determined, since it will be necessary to compute the strong match value $(8, 11, 2)$.

k=0													
i	0	1	2	3	4	5	6	7	8	9	10	11	12
j	A	B	A	A	D	A	C	B	A	A	B	C	
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1 C	0	0	0	0	0	0	1	1	1	1	1	1	1
2 B	0	0	1	1	1	1	1	2	2	2	2	2	2
3 C	0	0	1	1	1	1	1	2	2	2	2	2	3
4 B	0	0	1	1	1	1	1	2	3	3	3	3	3
5 D	0	0	1	1	1	2	2	2	3	3	3	3	3
6 A	0	1	1	2	2	2	3	3	4	4	4	4	4
7 A	0	1	1	2	3	3	3	3	4	5	5	5	5
8 D	0	1	1	2	3	4	4	4	4	5	5	5	5
9 C	0	1	1	2	3	4	4	5	5	5	5	5	6
10 D	0	1	1	2	3	4	4	5	5	5	5	5	6
11 B	0	1	2	2	3	4	4	5	6	6	6	6	6
12 A	0	1	2	3	3	4	5	5	6	7	7	7	7

k=1													
i	0	1	2	3	4	5	6	7	8	9	10	11	12
j	A	B	A	A	D	A	C	B	A	A	B	C	
0	-	-	-	-	-	-	-	-	-	-	-	-	-
1 C	-	-	-	-	-	-	1	1	1	1	1	1	1
2 B	-	-	-	-	-	-	1	2	2	2	2	2	2
3 C	-	-	-	-	-	-	2	2	2	2	2	2	3
4 B	-	-	-	-	-	-	2	3	3	3	3	3	3
5 D	-	-	-	-	-	-	2	3	3	3	3	3	3
6 A	-	-	-	-	-	-	2	3	4	4	4	4	4
7 A	-	-	-	-	-	-	2	3	4	5	5	5	5
8 D	-	-	-	-	-	-	2	3	4	5	5	5	5
9 C	-	-	-	-	-	-	5	5	5	5	5	5	6
10 D	-	-	-	-	-	-	5	5	5	5	5	5	6
11 B	-	-	-	-	-	-	5	6	6	6	6	6	6
12 A	-	-	-	-	-	-	5	6	7	7	7	7	7

k=2													
i	0	1	2	3	4	5	6	7	8	9	10	11	12
j	A	B	A	A	D	A	C	B	A	A	B	C	
0	-	-	-	-	-	-	-	-	-	-	-	-	-
1 C	-	-	-	-	-	-	-	-	-	-	-	-	-
2 B	-	-	-	-	-	-	-	2	2	2	2	2	2
3 C	-	-	-	-	-	-	-	2	2	2	2	2	3
4 B	-	-	-	-	-	-	-	3	3	3	3	3	3
5 D	-	-	-	-	-	-	-	3	3	3	3	3	3
6 A	-	-	-	-	-	-	-	3	4	4	4	4	4
7 A	-	-	-	-	-	-	-	3	4	5	5	5	5
8 D	-	-	-	-	-	-	-	3	4	5	5	5	5
9 C	-	-	-	-	-	-	-	3	4	5	5	5	6
10 D	-	-	-	-	-	-	-	6	6	6	6	6	6
11 B	-	-	-	-	-	-	-	6	6	6	6	6	6
12 A	-	-	-	-	-	-	-	6	7	7	7	7	7

Masks			
Y_A	Y_B	Y_C	Y_D
0	1	0	0
0	1	0	0
0	0	0	1
1	0	0	0
1	0	0	0
0	0	0	1
0	0	0	1
0	1	0	0
1	0	0	0
1	0	0	0

$i = 7$	
$M(7, 1, 1) = 1$	
$V_2 \quad C_2 = 0$	1
$M(7, 3, 1) = 2$	
$V_4 \quad C_4 = 0$	1
$M(7, 9, 1) = 5$	
$V_{10} \quad C_{10} = 0$	1
$M(7, 10, 1) = 5$	

$i = 8$	
$M(8, 1, 1) = 1$	
$V_2 \quad C_2 = 1$	0
$M(8, 3, 1) = 2$	
$V_4 \quad C_4 = 1$	0
$M(8, 9, 1) = 5$	
$V_{10} \quad C_{10} = 1$	1

$i = 9$	
$M(9, 1, 1) = 1$	
$V_2 \quad C_2 = 1$	0
$M(9, 3, 1) = 2$	
$V_4 \quad C_4 = 2$	0
$M(9, 9, 1) = 5$	
$V_{10} \quad C_{10} = 2$	1

$i = 10$	
$M(10, 1, 1) = 1$	
$V_2 \quad C_2 = 1$	0
$M(10, 3, 1) = 2$	
$V_4 \quad C_4 = 3$	0
$M(10, 9, 1) = 5$	
$V_{10} \quad C_{10} = 2$	1
$M(10, 10, 1) = 5$	

$i = 12$	
$M(12, 1, 1) = 1$	
$V_2 \quad C_2 = 1$	0
$M(12, 3, 1) = 3$	
$V_4 \quad C_4 = 2$	1
$M(12, 9, 1) = 6$	
$V_{10} \quad C_{10} = 1$	1

Figure 13. Example of the strip method for $k = 1$. The sequences are: $A = \text{ABAADACBAABC}$, $B = \text{CBCBDAADCDBA}$, $P = \text{CBB}$. Upper part shows the contents of matrix M . (This is only for comparison, since a complete matrix M is not computed by the algorithm.) The frames in bottom part show the actually made computations. The black boxes (bottom part) denote strong matches. The light gray boxes denote the cells at boundaries of strips computed explicitly. The dark gray boxes mean the cells computed explicitly for the next level.

Extension to ComputeStrip algorithm

```

11   $T(i, k) \leftarrow T(i, k) \mid (V_{j_0} \ll j_0)$ 
12  if  $M(i-1, j_1, k) = M(i, j_1, k)$  then
13     $T(i, k) \leftarrow T(i, k) \& \sim(1 \ll j_1)$ 

```

Figure 14. Extension to the strip computing algorithm

Extension to ComputeLevel algorithm

```

18.1   $T(i, k) \leftarrow T(i, k) \& \sim(1 \ll j^*)$ 

```

Figure 15. Extension to the level computing algorithm

Frame $i = 8$ shows the computations as follows. $M(8, 1, 1)$ is obtained in a preliminary computation of the top boundary ((8, 1, 1) is not a strong match so the value is copied from its left neighbor). Vector $V_2(8)$ is computed in a bit-parallel way (Figure 9, lines 01–04) according to $M(8, 1, 1)$ and $V_2(7)$. Since (8, 3, 1) is not a match, its value is obtained as maximum of $M(7, 3, 1)$ and $M(7, 1, 1) + C_2(8)$. Similarly, $V_4(8)$ is computed in a bit-parallel way on $M(8, 3, 1)$ and $V_4(7)$. Then, $M(8, 9, 1)$ is determined. Finally, $V_{10}(8)$ is computed. In this column, there was no need to compute other values of M since no strong matches in column 9 at level 2 exist.

In a similar way, the computations proceed in the remaining frames.

5.3. Obtaining a Requested Sequence

The algorithm introduced in the previous subsection computes only the CLCS length, but sometimes also the sequence can be requested. Adding such a support is, however, rather straightforward. An idea is similar to that of BP algorithm for the LCS problem.

We construct a traceback bit array T of sizes $(n+1) \times (r+1) \times (m+1)$ bits. For clearer presentation we neglect here the implementation details and only note that in fact the 3-dimensional bit array is implemented as 2-dimensional array of sizes $(n+1) \times (r+1)$ containing in each cell $\lceil (m+1)/w \rceil$ -word vector of computer words. Array T is filled by extended ComputeStrip and ComputeLevel functions as presented in Figures 14 and 15. A general idea is to store in $T(i, j, k)$ a bit that acts as a “signpost” to which neighbor bit we should go when traversing this array. If $T(i, j, k) = 0$ we move leftwards, otherwise upwards. This bit is unimportant when we have a match, since if the match is not strong we move to $T(i-1, j-1, k)$, while for the strong one we follow to $T(i-1, j-1, k-1)$.

The matches are handled by a traceback algorithm in a special way, so we neglect them for a while (in this paragraph). The strip computing function saves bit vector V in T (line 11), since if $T(i, j, k) = 0$ (when we are not at strip boundary) we know that $M(i, j, k) > M(i, j-1, k)$ so we must go leftwards to the equal value of $M(i, j, k)$. Otherwise we go upwards as $M(i, j-1, k) = M(i, j, k)$. A bit more care is necessary when we are at row being a bottom edge of a strip. An information about a difference between cells from vertically-adjacent strips are not stored in bit vector V , so we must handle this situation in some other way. Let us remember that the values of M for bottom edges of trips are known explicitly, so we can simply compare $M(i-1, j_1, k)$ and $M(i, j_1, k)$ to set a “signpost” bit $T(i, j_1, k)$. If the pointed

ComputeSequence()

```

01   $\ell^* \leftarrow \ell; \quad k \leftarrow r; \quad i \leftarrow n; \quad j \leftarrow m$ 
02  while  $\ell > 0$  do
03      if  $a_i = b_j$  then
04          if  $k > 0$  and  $a_i = p_k$  then
05               $k \leftarrow k - 1$ 
06               $s_\ell \leftarrow a_i; \quad \ell^* \leftarrow \ell^* - 1$ 
07               $i \leftarrow i - 1; \quad j \leftarrow j - 1$ 
08          else
09              if  $j$ th bit of  $T(k, i)$  is 1 then
10                   $j \leftarrow j - 1$ 
11              else
12                   $i \leftarrow i - 1$ 
13  return  $s_1 s_2 \dots s_\ell$ 

```

Figure 16. Traceback procedure to obtain a constrained longest common subsequence

values are equal we can go safely leftwards, otherwise the upper neighbor $M(i, j_1 - 1, k)$ must be equal to $M(i, j_1, k)$ so we store 1 bit in $T(i, j_1, k)$.

The discussion in the above paragraph was for the case when the current cell is not a match. The bit values in T are wrong for matches since we move diagonally (not to the left or to the upper cell), but this is unimportant since the matches are detected by comparing the symbols of the sequence. The algorithm finding a CLCS according to the above description is presented in Figure 16. Each time the actual triple represents a match (strong or not) the matching symbol is output. An example of the computations made by ComputeSequence algorithm is given in Figure 17.

5.4. Implementation Details and Complexity Analysis

A concept of the proposed CLCS-length-computing algorithm suggests that the space complexity is $O(rnm)$ due to matrix M . Since we only compute the CLCS length, storing two levels of the matrix, k th and $(k - 1)$ th, suffices. Moreover, from the $(k - 1)$ th level only the values of $M(i - 1, j - 1, k - 1)$ for which (i, j, k) is a strong match are necessary. Let us denote the number of strong matches at k th level by d_k (if $k \leq 0$ or $k > r$ then we define $d_k = 0$). These matches can be stored in an FIFO queue, since the values are computed in the same order as they are needed at the next level. Also, since the level is computed in a column-wise manner, only the bit vectors of two columns i th and $(i - 1)$ th are necessary to store. Finally, the bit masks are needed for all alphabet symbols. Therefore, the total space complexity of the method is:

$$O(\sigma \lceil m/w \rceil) + O(\lceil m/w \rceil + v_k) + O(d_{k-1} + d_k + d_{k+1}) = O(\sigma \lceil m/w \rceil + v' + d'), \quad (3)$$

where $v_k = |\{b_j = p_k : 0 < j \leq m\}|$ at level k (by definition $v_0 = v_{k+1} = 0$), $v' = \max_{0 \leq k \leq r} v_k$, and $d' = \max_{0 \leq k \leq r} d_k$.

Let us now consider the time complexity of this method. The number of strips in a single column at k th level is $O(v_k)$, but the strips larger than w are emulated as arrays of computer words, so the total number of computer words representing vectors V is $O(v_k + \lceil m/w \rceil)$. For each computer word (containing a strip or its part) it is necessary to make $O(1)$ bit-parallel computations per column. Moreover,

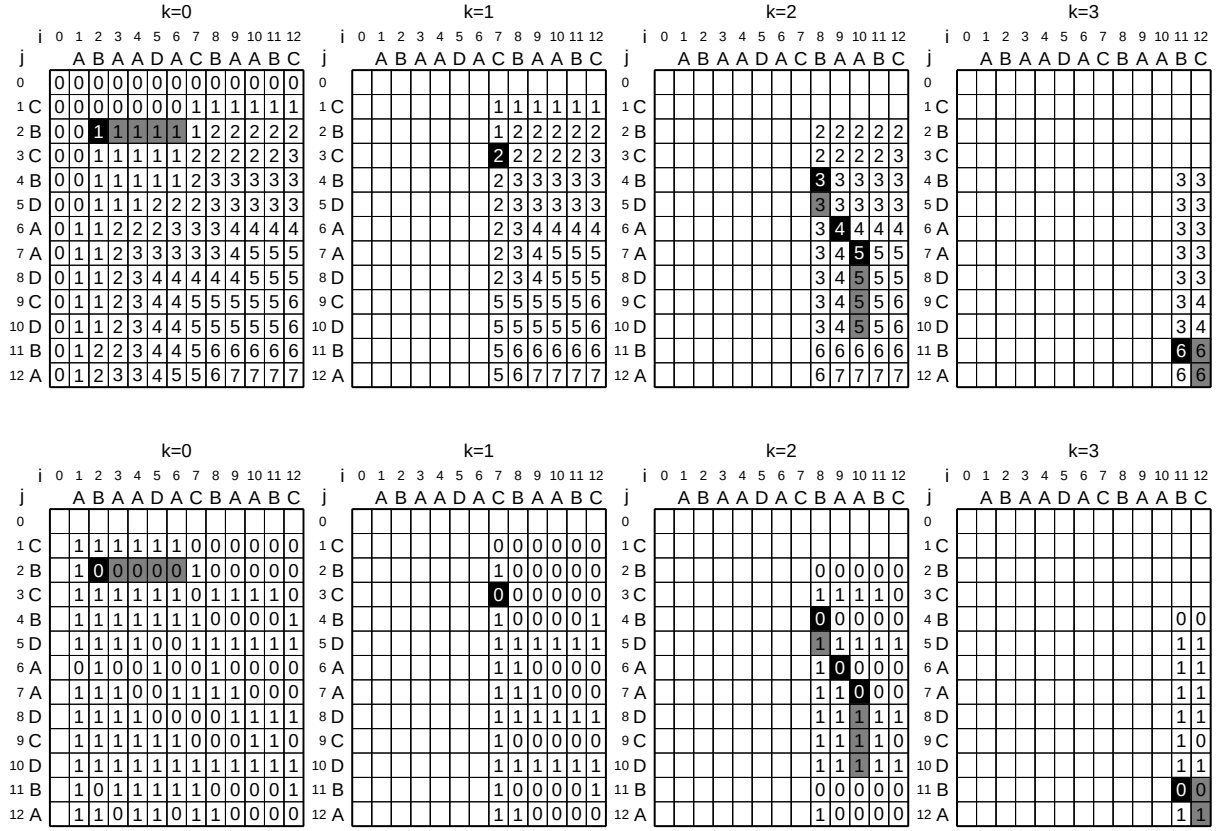


Figure 17. Example of the traceback procedure. The sequences are: $A = \text{ABAADACBAABC}$, $B = \text{CBCBDAADCDBA}$, $P = \text{CBB}$. Upper part shows (only for comparison) the contents of the matrix M . The bottom part shows the contents of the traceback array T . The black boxes denote matches taken to the result. The gray boxes denote the visited. The output sequence is BCBAAB (underlined symbols form constrained sequence)

there are $O(h_{k+1})$ columns for which there is a need to compute the M values for the $(k+1)$ th level ($h_k = |\{a_i = p_k : 0 < i \leq n\}|$, by definition $h_0 = h_{r+1} = 0$). These computations take $O(m)$ time per each such column. There are also removals of the least significant 0s from vectors V in the columns containing strong matches. For each of h_k such columns the total cost of removing 0s is $O(m)$. From the above, the total time complexity of the proposed algorithm is:

$$\sum_{k=0}^r O(mh_k + n(v_k + \lceil m/w \rceil) + mh_{k+1}) = \sum_{k=0}^r O(mh_k + nv_k + n\lceil m/w \rceil). \quad (4)$$

In the worst-case, $h_k = n$, $v_k = m$, for all $0 < k \leq r$, so the time complexity is $O(rnm)$. The worst-case time complexity can be also expressed in terms of the total number of strong matches, $D = \sum_{k=0}^r d_k$. In this case, $v_k h_k = d_k$, so the time complexity is:

$$\sum_{k=0}^r O\left(m \frac{d_k}{v_k} + nv_k + n\lceil m/w \rceil\right), \quad (5)$$

which is maximized for $v_k = \Theta(\sqrt{m/n}\sqrt{d_k})$ for all valid k . In this case we obtain the bound:

$$\sum_{k=0}^r O\left(m \frac{d_k}{\sqrt{m/n}\sqrt{d_k}} + n\sqrt{m/n}\sqrt{d_k} + n\lceil m/w \rceil\right) = \quad (6)$$

$$\sum_{k=0}^r O\left(\sqrt{mn}\sqrt{d_k} + \sqrt{mn}\sqrt{d_k} + n\lceil m/w \rceil\right) = \quad (7)$$

$$O\left(\sqrt{mn} \sum_{k=0}^r \sqrt{d_k} + rn\lceil m/w \rceil\right). \quad (8)$$

For fixed D , the sum $\sum_{k=0}^r \sqrt{d_k}$ is maximized if all d_k are equal for $0 < k \leq r$. Therefore, the worst-case time complexity can be expressed as:

$$O\left(\sqrt{mn} \sum_{k=0}^r \sqrt{D/r} + rn\lceil m/w \rceil\right) = O\left(\sqrt{rnmD} + rn\lceil m/w \rceil\right). \quad (9)$$

The total number of strong matches can be at most $D = rnm$, but in practice it is much smaller.

To determine the average-case time complexity of the algorithm we will assume that all three sequences (A , B , and P) are generated by uniform random generator and the alphabet size is σ . We will start from the bound obtained in Equation 4. The sequences are independent, so there is no dependency between the number of strong matches for different levels. Moreover, there is no dependency between v_k and h_k for a single level. Therefore, to obtain the average-case time complexity we can calculate the expected value of each term of the sum.

Let us now consider the first term, mh_k , for some $0 < k \leq r$. The value h_k is the number of symbols of A equal p_k . Obviously, the probability that $a_i = p_k$ is $1/\sigma$ and the expected value of h_k is n/σ . Similarly, the expected value of v_k is m/σ . Therefore the average-case time complexity of the algorithm is:

$$O(n\lceil m/w \rceil) + \sum_{k=1}^r O(mn/\sigma + nm/\sigma + n\lceil m/w \rceil) = O\left(rn \left\lceil \frac{m}{\min\{\sigma, w\}} \right\rceil\right). \quad (10)$$

If we are interested not only in the CLCS length we need to store a traceback array occupying $O(rn\lceil m/w \rceil)$ words of space, which is a dominant term. The time to compute this array is the same as in the CLCS-length-computing method. The cost of obtaining a requested sequence is small since in each loop iteration of the `ComputeSequence` algorithm we decrease at least one variable, so the worst- and average-case time of computation is the sum of sequence lengths:

$$O(r + n + m) = O(n). \quad (11)$$

6. Application of the Algorithm in the Multiple CLCS Problem

In some cases, comparing only two sequences is insufficient if we are interested in a longest common subsequence for a set of sequences. Such a situation occurs for example in bioinformatics. A dynamic-programming-based algorithm for the pairwise LCS problem easily extends to the case of a set of equally-

length sequences $\mathcal{S} = \{S^1, S^2, \dots, S^N\}$.² It suffices to construct an N -dimensional matrix of sizes $(z+1) \times (z+1) \times \dots \times (z+1)$ and fill it according to the modified recurrence of (1):

$$M([i_1, \dots, i_N]^T) = \begin{cases} 0 & \text{for } i_k = 0 \text{ where } 1 \leq k \leq N, \\ \max\{M([i_1 - d_1, \dots, i_N - d_N]^T)\} & \text{for } 0 < i_k \leq z, 0 \leq d_k \leq 1, \\ & 0 < \sum_k d_k < N, \text{ where } 1 \leq k \leq N, \\ M([i_1 - 1, \dots, i_N - 1]^T) + 1 & \text{for } 0 < i_k \leq z, s_{i_1}^1 = s_{i_2}^2 = \dots = s_{i_N}^N, \\ & \text{where } 1 \leq k \leq N. \end{cases} \quad (12)$$

The time complexity of this algorithm is $O(2^N z^N)$. The second term is due to the number of cells in the matrix, while the first is a result of the second case in (12) in which a maximum over $2^N - 2$ neighbors of the current cell is computed. If N is unbounded this is an exponential complexity, which means that computing the whole DP matrix even for the moderate number of sequences of reasonable sizes is impossible in practice. Unfortunately, no much better algorithm exists to the multiple LCS problem and it is known that the problem is NP-hard [22]. One of the fastest exact methods for the multiple LCS (M-LCS) problem is related on the sparsity of the dynamic programming matrix [29], but it is still exponential-time.

Pairwise/multiple sequence alignment (PSA/MSA) problems are closely related to the LCS/M-LCS problems and algorithms solving the PSA/MSA problems can be easily adopted to solve the LCS/M-LCS problems. Frequently, the same holds in the opposite way. Moreover, the two families of problems are equivalent if the score function in the PSA/MSA problems is of some special form, e.g., a match score is 1, while an insertion and deletion score is 0. In general, in the exact MSA-solving algorithms the same N -dimensional matrix is constructed and is filled by some similar recurrence.

Since, the M-LCS and MSA problems are so hard and rarely may be solved exactly, many heuristics have been proposed for them. The heuristics often start from computation of the LCS (PSA) for all $N(N-1)/2$ pairs of the sequences from $\mathcal{S}$ and then follow in various ways. One possibility is to replace two most similar sequences, i.e., those for which an LCS is the longest (the score of alignment is the largest) by its LCS (alignment) and repeat this process $N-2$ times obtaining finally a single subsequence (alignment). This heuristic is not, of course, optimal, so the obtained subsequence is not the longest possible (the alignment is not of the maximal score), but the results are often quite well, so we can agree for the imperfectness of the results due to fast computations. A good review of the approaches for solving the MSA problem can be found in [13] and the same ideas can be used for the M-LCS problem.

A multiple constrained longest common subsequence (M-CLCS) problem is a generalization of the M-LCS problem, which of course is at least equally hard (when a constrained sequence is empty the M-CLCS problem reduces to the M-LCS problem). Therefore, methods for the M-LCS problem were generalized to solve the M-CLCS problem [26]. Firstly, the pairwise constrained longest common subsequences for all pairs of sequences from the set are computed. Then, in some way an order of merging the sequences is determined. Finally, the sequences are merged according to this order, using the CLCS-computing algorithm. In these approaches, any pairwise CLCS-computing algorithm may be used, i.e. the BP method introduced in this paper.

²We assume that the sequence are of equal length z only for simplicity of discussion, and relaxing from this constraint is straightforward

7. Experimental Results

We performed two series of experiments to evaluate the proposed algorithm in various scenarios. In all situations, the times were obtained as medians of 501 executions on a computer equipped with AMD Phenom II X4 810 CPU (2.6 GHz real CPU clock) with $w = 64$. The algorithms were implemented in C++ and compiled with MS Visual C++ 2008 compiler (maximum optimization for speed was set in the compiler options).

The experimental comparison of the CLCS-computing algorithms by Deorowicz and Obstój [11] (for the pairwise case) showed that the fastest algorithms for the problem are the ones by Chin *et al.* [6] and by Deorowicz [10], both with the entry-exit points improvement [11]. In this paper, we make similar tests using the same source codes, so we refrained from including in the comparison the other methods, mainly due to the clarity of figures. Therefore, the compared algorithms are:

- ChinEE—the version of Chin *et al.* algorithm [6] with the entry-exit points improvement [11],
- DeoEE—the version of Deorowicz algorithm [10] with the entry-exit points improvement [11],
- BPEE-32—the bit-parallel strip algorithm with the entry-exit points ($w = 32$) (Section 5.2),
- BPEE-64—the bit-parallel strip algorithm with the entry-exit points ($w = 64$) (Section 5.2).

In the first series of experiments, we compared the CLCS-length- and CLCS-computing algorithms on random data. The results are shown in Figure 18. For better clarity, instead of execution times, we present how many times the algorithm is faster than the method by Chin *et al.* improved by introduction of the entry-exits points [11]. The sequences were produced using a uniform random generator.

In the first test of this series, an influence of the alphabet size on the computation times is measured (Figures 18a, 18b, 18d, 18e). This is motivated by the fact, that roughly the larger the alphabet, the smaller number of strong matches and the larger the strips. Of course a word size is some barrier, since too large strips must be emulated as vectors of computer words. For the length-only case when the main sequences are of significantly different lengths and the constrained sequence is short ($r = 4$), the bit-parallel version presented in this paper outperforms the other methods significantly (Figure 18a). If also the sequence is requested (Figure 18d) bit-parallel algorithms are still significantly faster than the other. In both cases (length-only and sequence-also), for the smallest possible alphabet ($\sigma = 2$) the gain is negligible, but when the alphabet becomes larger, the gain over the ChinEE algorithm achieves the factor 13–16 ($\sigma = 20$). For small alphabets the 32- and 64-bit versions are of similar speed, since rarely the strips do not fit in 32-bit words. For moderate alphabets, $20 \leq \sigma \leq 100$, the gain of the 64-bit version is noticeable (about 35%). Interestingly, for even larger alphabets, the algorithms become of equal speed due to the fact that for such large alphabets the constrained sequence restricts the resulting sequence so significantly that the part of matrix M to compute is very small; often the resulting sequence does not exist, which can be determined very fast. The algorithm BPEE-64 for moderate alphabets, $20 \leq \sigma \leq 100$, is also a few times faster than the fastest method known to date (DeoEE).

In Figures 18b and 18e, the constrained sequence is longer (12 symbols) and the main sequences are of equal lengths. The results are similar as in the first experiment but the gains of bit-parallel methods are a bit smaller. For binary alphabet, the ChinEE method is the fastest.

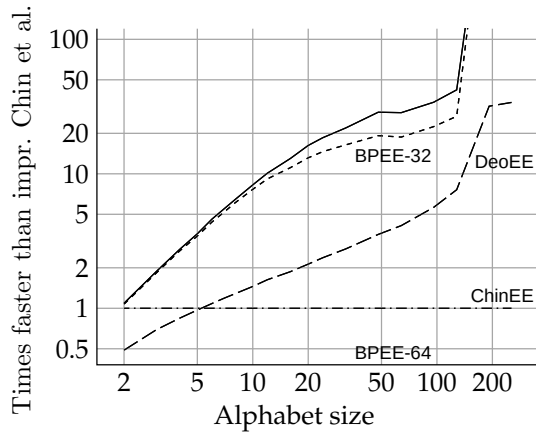
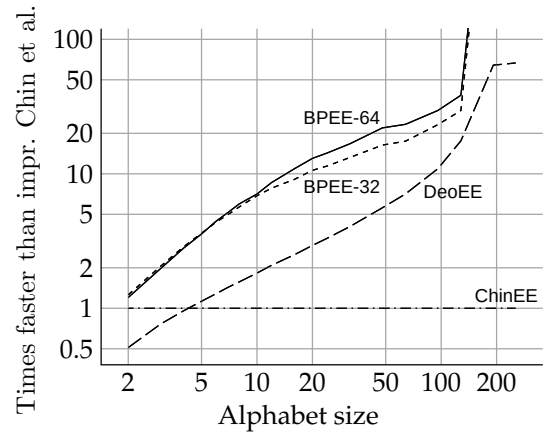
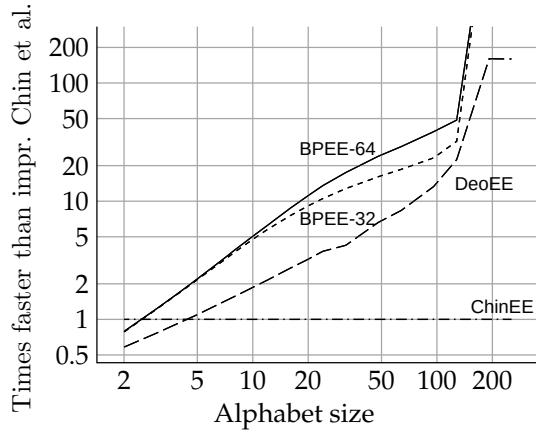
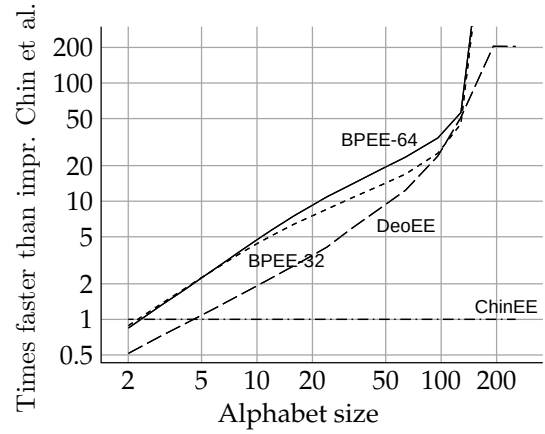
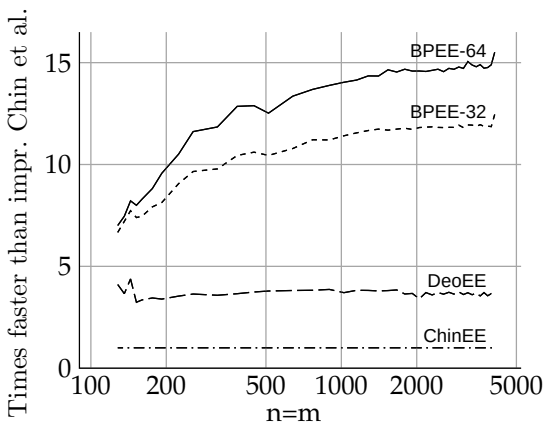
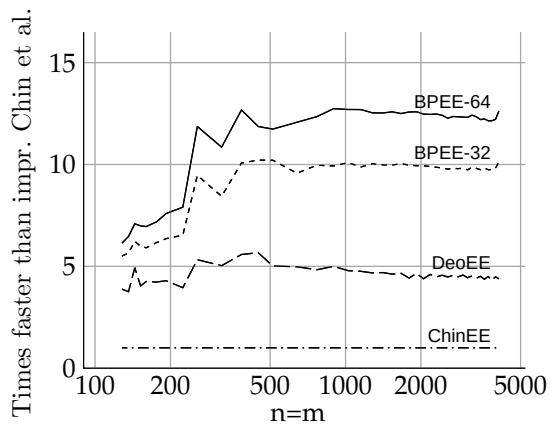
(a) $m = 512, n = 4096, r = 4$, length only(d) $m = 512, n = 4096, r = 4$, sequence also(b) $m = 2048, n = 2048, r = 12$, length only(e) $m = 2048, n = 2048, r = 12$, sequence also(c) $r = 4, \sigma = 20$, length only(f) $r = 4, \sigma = 20$, sequence also

Figure 18. Experimental results of the CLCS-length- (left column) and CLCS- (right column) computing algorithms

Data Set	Num. Seq.	Contents	Seq. Length (min, med, max)	Alph. Size	H_0	Origin
ds0	7	H-RNase3, H-RNase2, BP-RNaseA, BS-RNase, H-RNaseA, H-RNase4, RC-RNase	(111,124,134)	20	4.172	[7]
ds1	6	gi 119124 sp P12724 ecp_human, gi 2500564 sp P70709 ecp_rat, gi 13400006 pdb ldyt, gi 20930966 ref xp_142859.1, gi 20873960 ref xp_127690.1, gi 20930966 ref xp_142859.1	(124,149,185)	20	4.246	[7]
ds2	6	gi 20930966 ref XP_142859.1, gi 119124 sp P12724 ECP_HUMAN, gi 2500564 sp P70709 ECP_RAT, gi 13400006 pdb, gi 20930966 ref XP_142859.1, gi 20873960 ref XP_127690.1	(131,142,160)	20	4.189	[7]
ds3	5	gi 10068295 gb AAE40716.1, gi 17549935 ref NP_510780.1, gi 28509297 ref XP_282983.1, gi 28499937 ref XP_204162.2, gi 4902995 dbj BAA77929.1	(189,277,327)	20	4.191	[7]
ds4	6	Protease: HTLV-1, RSV, HIV-1, SRV-1, CaMV, 17.6	(98,114,123)	20	4.111	[21]

Table 2. Data sets of real data used in the experiments [11]. Column H_0 contains 0-order entropy of the sequences (in bits). Note that the entropy of uniform random sequence over alphabet of size 20 is about 4.322 bits

The results of the last experiment in this series (Figures 18c and 18f) show how the speedup changes when both sequences become longer ($\sigma = 20$, $r = 4$, equal-length main sequences). For very short sequences the gain is less than 10-fold, but when the sequences become longer than about 1000, the gain becomes steady, around 15-fold for 64-bit version and 12-fold for 32-bit version, in both length-only and sequence-also variants.

In the second series of experiments, the algorithms are evaluated on real data of RNase sequences of lengths from range [111, 327]. The data sets characterized in Table 2 are the same as in [11, Table 2]. An alphabet size for all of them is 20. The methodology of experiments is the same as in [11], so for each data set CLCS length and CLCS are computed for all pairs within a dataset and total times of computations is calculated. A motivation for such experiments is that in various methods for multiple CLCS problem the first step is to compute the pairwise CLCS for all pairs within a set of sequences. Usually, in next steps also pairwise CLCS for some pairs of sequences is necessary.

The results of this experiment are shown in Table 3. It can be observed that the algorithm BPPE-64 is the fastest one in all examined cases. It is from 5.6 to 16.7 times faster than ChinEE. Its advantage over the fastest to date method (DeoEE) is from 2 to 4-fold. These results show that the proposed method can be used to significantly speedup both pairwise and multiple sequence variants of the constrained longest common subsequence problem.

Table 3. CLCS-length and CLCS computation times (in ms) of algorithms on real data. In parenthesis: the number of times the given algorithm is faster than ChinEE

Data set	Constrained sequence	CLCS length only				CLCS also			
		ChinEE	DeoEE	BPEE-32	BPEE-64	ChinEE	DeoEE	BPEE-32	BPEE-64
ds0	HKH	3.078 (1.000)	0.922 (3.338)	0.395 (7.792)	0.336 (9.161)	3.871 (1.000)	1.117 (3.466)	0.588 (6.583)	0.486 (7.965)
ds1	HKH	4.333 (1.000)	1.310 (3.308)	0.539 (8.039)	0.437 (9.915)	5.442 (1.000)	1.567 (3.473)	0.825 (6.596)	0.668 (8.147)
ds1	HKSH	3.917 (1.000)	1.159 (3.380)	0.534 (7.335)	0.472 (8.299)	5.030 (1.000)	1.423 (3.535)	0.817 (6.157)	0.712 (7.065)
ds1	HKSTH	3.734 (1.000)	1.128 (3.310)	0.549 (6.801)	0.497 (7.513)	4.831 (1.000)	1.393 (3.468)	0.831 (5.813)	0.733 (6.591)
ds2	HKSH	2.940 (1.000)	0.869 (3.383)	0.368 (7.989)	0.336 (8.750)	3.745 (1.000)	1.070 (3.500)	0.562 (6.664)	0.481 (7.786)
ds2	HKSTH	2.526 (1.000)	0.720 (3.508)	0.341 (7.408)	0.305 (8.282)	3.331 (1.000)	0.915 (3.640)	0.507 (6.570)	0.441 (7.553)
ds3	HKH	8.328 (1.000)	2.389 (3.486)	0.745 (11.179)	0.593 (14.044)	15.676 (1.000)	2.878 (5.447)	1.205 (13.009)	0.938 (16.712)
ds4	DGGG	1.986 (1.000)	0.645 (3.079)	0.333 (5.964)	0.307 (6.469)	2.502 (1.000)	0.791 (3.163)	0.487 (5.138)	0.447 (5.597)

8. Conclusions

There are several algorithms solving the CLCS problem. They are based on classical dynamic programming or Hunt–Szymanski paradigm. We proposed a bit-parallel algorithm computing a CLCS or only its length. Its worst-case time complexity is $O(rnm)$ and the average-case complexity is $O(rnm/\sigma + rn\lceil m/w \rceil)$. The worst-case time complexity can be also expressed as $O(\sqrt{rnmD} + rn\lceil m/w \rceil)$, where D is the total number of strong matches. Practical experiments were performed on a 64-bit processor on random sequences over alphabets of various sizes (but mostly the alphabet size was 20 to emulate the case of protein comparison) and on real data. They show that the bit-parallel method proposed in the paper is by far the fastest algorithm computing a CLCS or only its length for almost all examined cases. For a typical case ($\sigma = 20$), the gain over the Chin *et al.* method with the entry-exit points was 10–16-fold. The 32-bit version is slower typically by about 20–30% than the 64-bit version. Moreover, the 64-bit version of the proposed algorithm is about 2–7 times faster than the currently fastest CLCS computing algorithm by Deorowicz and Obstójk.

The main drawback of the presented algorithm is that the time complexity is dependent on the data, especially on the alphabet size. Therefore, the theoretical speedup in the average case is limited to $\min\{w, \sigma\}$ rather than w , which is achievable in the LCS problem. The open question related to bit-parallelization of the CLCS algorithms is whether this drawback can be fixed. Concerning the CLCS

problem in general, the important question appears to be whether the $O(rnm)$ worst-case time complexity can be improved or not.

Acknowledgements

The author thanks Zbigniew J. Czech, Szymon Grabowski, and Roman Starosolski for comments on the paper. The research of this project was supported by the Minister of Science and Higher Education grant 3177/B/T02/2008/35.

References

- [1] Allison, L., Dix, T.: A bit-string longest common subsequence algorithm, *Information Processing Letters*, **23**, 1986, 305–310.
- [2] Apostolico, A.: *General pattern matchings*, chapter 13, Handbook of Algorithms and Theory of Computation, Atallah, M.J., 1998.
- [3] Arslan, A., Egecioğlu, O.: Algorithms for the constrained longest common subsequence problems, *International Journal of Foundations of Computer Science*, **16**(6), 2005, 1099–1109.
- [4] Baeza-Yates, R., G.H., G.: A new approach to text searching, *Communications of the ACM*, **35**(10), 1992, 74–82.
- [5] Bergroth, L., Hakonen, H., Raita, T.: A survey of longest common subsequence algorithms, *Proceedings of 7th International Symposium on String Processing Information Retrieval (SPIRE)*, Curuña, Spain, 2000.
- [6] Chin, F., De Santis, A., Ferrara, A., Ho, N., Kim, S.: A simple algorithm for the constrained sequence problems, *Information Processing Letters*, **90**, 2004, 175–179.
- [7] Chin, F., Ho, N., Lam, T., Wong, P.: Efficient Constrained Multiple Sequence Alignment with Performance Guarantee, *Journal of Bioinformatics and Computational Biology*, **3**(1), 2005, 1–18.
- [8] Crochemore, M., Landau, G., Ziv-Ukelson, M.: A sub-quadratic sequence alignment algorithm for unrestricted cost matrices, *SIAM Journal of Computing*, **32**(6), 2003, 1654–1673.
- [9] Crochemore, M., Iliopoulos, C., Pinzon, Y., Reid, J.: A fast and practical bit-vector algorithm for the longest common subsequence problem, *Information Processing Letters*, **80**, 2001, 279–285.
- [10] Deorowicz, S.: Fast Algorithm for Constrained Longest Common Subsequence Problem, *Theoretical and Applied Informatics*, **19**(2), 2007, 91–102.
- [11] Deorowicz, S., Obstó, J.: Constrained Longest Common Subsequence Computing Algorithms in Practice, Accepted for publication in Computing and Informatics, 2010, (http://sun.aei.polsl.pl/~sdeor/pub/tr_do2009.pdf).
- [12] Dömölki, B.: An algorithm for syntactical analysis, *Computational Linguistics*, **3**, 1964, 29–46.
- [13] Gotoh, O., Yamada, S., Yada, T.: Multiple Sequence Alignment, in: *Handbook of Computational Molecular Biology* (S. Aluru, Ed.), chapter 3, Chapman & Hall/CRC, 2006.
- [14] Gusfield, D.: *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [15] Hirschberg, D.: An information-theoretic lower bound for the longest common subsequence problem, *Information Processing Letters*, **7**(1), 1978, 40–41.

- [16] Hunt, J., Szymanski, T.: A fast algorithm for computing longest common subsequences, *Communications of the ACM*, **20**(5), 1977, 350–353.
- [17] Hyyrö, H.: Bit-Parallel LCS-length Computation Revisited, *Proceedings of the 15th Australian Workshop on Combinatorial Algorithms (AWOCA)*, University of Sydney, Australia, 2004.
- [18] Iliopoulos, C. S., Rahman, M. S.: New Efficient Algorithms for LCS and Constrained LCS Problem, *Information Processing Letters*, **106**(1), 2008, 13–18.
- [19] Kuo, S., Cross, G.: An Improved Algorithm to Find the Length of the Longest Common Subsequence of Two Strings, *ACM SIGIR Forum*, **23**(3-4), 1989, 89–99.
- [20] Levenshtein, V.: Binary codes capable of correcting deletions, insertions, and reversals, *Problems in Information Transmission*, **1**, 1965, 8–17.
- [21] Lu, C., Huang, Y.: A Memory-Efficient Algorithm for Multiple Sequence Alignment with Constraints, *Bioinformatics*, **21**(1), 2005, 20–30.
- [22] Maier, D.: The Complexity of Some Problems on Subsequences and Supersequences, *Journal of the ACM*, **25**(2), 1978, 322–336.
- [23] Masek, W., Paterson, M.: A faster algorithm computing string edit distances, *Journal of Computer System Science*, **20**(1), 1980, 18–31.
- [24] Peng, C.-L.: *An Approach for Solving the Constrained Longest Common Subsequence Problem*, Master Thesis, Department of Computer Science and Engineering, National Sun Yat-sen University, Taiwan, 2003, <http://etd.lib.nsysu.edu.tw/ETD-db/ETD-search/getfile?URN=etd-0828103-125439&filename=etd-0828103-125439.pdf>.
- [25] Peng, Z., Ting, H.: Time and Space Efficient Algorithms for Constrained Sequence Alignment, *Proceedings of the Implementation and Application of Automata, 9th International Conference (CIAA)*, 2004.
- [26] Shyu, S., Tsai, C.-Y.: Finding the longest common subsequence for multiple biological sequences by ant colony optimization, *Computers and Operations Research*, **36**(1), 2009, 73–91.
- [27] Tank, C., Lu, C., Chang, M.-T., Tsai, Y.-T., Sun, Y.-J., Chao, K.-M., Chang, J.-M., Chiou, Y.-H., Wu, C.-M., Chang, H.-T., Chou, W.-I.: Constrained multiple sequence alignment tool development and its application to RNase family alignment, *Proceedings of the first IEEE Computer Society Bioinformatics Conference (CSB 2002)*, 2002.
- [28] Tsai, Y.-T.: The constrained common subsequence problem, *Information Processing Letters*, **88**, 2003, 173–176.
- [29] Wang, Q., Korkin, D., Shang, Y.: Efficient dominant point algorithms for the multiple longest common subsequence (MLCS) problem, *Proceedings of the 21st International Joint Conference on Artificial Intelligence* (H. Kitano, Ed.), Pasadena, California, USA, 2009.
- [30] Wang, W.-L.: *Longest Common Subsequence with Constraints*, Master Thesis, Department of Computer Science and Information Engineering National Chi-NanUniversity, R. O. C., June 2006, <http://alg.csie.ncnu.edu.tw/or/WLWang2006.pdf>.
- [31] Wong, C., Chandra, A.: Bounds for the string editing problem, *Journal of the ACM*, **23**(1), 1976, 13–16.